Notes on *Recursive Introduction to the Theory of Computation by Carl Smith [2]*

Kedar Mhaswade

30 June 2026

These notes are dedicated to the memory of Carl Smith (1950-2004).

Reflection 1 (Introduction)**.** These are the author's highly personal notes based on this book [2]. They may occasionally sound like author's conversation with the reader (or even himself). Even if they are personal and appear in the public domain, they may help the reader, although the author isn't exactly sure how. He wrote them for 0) the love of mathematics and writing 1) LaTeX practice (of writing hopefully readable mathematics, albeit longer than appearing in standard texts), and 2) reliable archival (paper, on which most work is created, tends to get lost) that incurs minimal overhead.

These notes are interspersed with "reflection boxes" like this one. They are meant to express author's 'rough' thinking process (mental catharsis of sorts), of which frustration is often an inseparable part. The author writes mostly in first person for an authentic conversational tone.

0.1 Preface

> *Audience, level, and treatment–a description of such matters is what prefaces are supposed to be about.*
>
> — Paul Halmos
> I Want to be a Mathematician [1]

This is a graduate-level textbook that can be covered in one semester. It defines 'computation' succinctly:

> *Computation is the **movement and transformation of 'data' through 'processors'**, which are understood as expressly designed systems made of well-defined components.*
>
> — Carl Smith

The book describes three "models of computation", each offering a different perspective. It also describes which functions are *computable* (emphasis mine):

> *Given the **firm knowledge of what is and what is not computable**, we move on to consider the complexity of computing the things that are computable.*
>
> — Carl Smith

About 'recursion' in the title of the book:

> *The title of this book contains the word 'recursion,' which has several meanings, many of which are appropriate. The original meaning of the word **recurse** was "a looking back." Indeed, in this book we look back at the fundamental results that shaped the theory of computation as it is today. The object of study for most of the book is the **partial recursive functions**, functions that derive their name from the way they are defined, using an operator that 'looks back' to prior function values. There is also a **recursion theorem**, which enables us to construct programs that can look back on their own code. In this treatment, recursion theorems are introduced relatively early and used often.*
>
> — Carl Smith

Reflection 2. For a while now, I wanted to understand the basic theory of computation better. In particular, I wanted to understand the equivalence between Turing machines and recursive functions as "models of computation." A cursory glance at the book finally made me commit to doing it along with other things I have committed to.

The textbook is about a fundamental computation itself. It may not be relevant in the age of faulty A.I., but I don't care. This was overlooked during my previous graduate days, and I wanted to try and fill that gap. There are several excellent books on this topic, but I will take a look at them later. For now, I have this book.

I may not be able to finish it in time, but try I shall.

Chapter 1

Models of Computation

> *The first artifact of computation that we will dismiss is the notion that arguments may be of different types. All our inputs, outputs, temporary variables, etc., will be natural numbers (members of* $\mathbb{N}$*). We will argue informally that* ***all the other commonly used argument types are included solely for convenience of programming and are not essential to computation****.*
>
> — Carl Smith

1.1 Random Access Machines

Smith provides a lucid description that we produce verbatim (emphasis is ours) here. This is the essence of all register-based, stored-program machines:

Box 1: RAM

All contemporary computers allow the 'random' accessing of their memory. An address is sent to the memory which returns the data stored at the given address. The name 'random access machine' stems from the fact that the earlier models were *not random access*; they were based on sequential tapes or they were functional in nature. We will consider these models later. As a starting point of our investigation, we will consider a model that strongly resembles assembly language programming on a conventional computer. The model that we introduce will perform very simple operations on registers.
Every real-world computer has a fixed amount of memory. This memory can be arbitrarily extended, at great loss of efficiency, by adding a tape or disk drive. Then the ultimate capacity of the machine is limited only by one's ability to manufacture or purchase tapes or disks. Not wanting to consider matters of efficiency just yet, our random access machine (RAM) will have a potentially infinite set of registers, $R_1, R_2, \ldots$, each capable of holding any natural number.
Notice that we have just eliminated main storage and peripheral storage (and their management) as artifacts of how we (necessarily) perform computations. **We will be concerned with data, and computation on that data**. The issues of input and output will not be addressed in any detail.
RAM programs will be abstractions of assembly language programs in the sense that they use a very limited set of instruction types and the *only control structure allowed is the much maligned branch instruction.* There is nothing special about our choice of instructions.
Among the possible choices for instruction sets, the one chosen here is based on *some nontechnical notion of simplicity.*
RAM programs are finite sequences of very basic instructions. Hence, each RAM program will reference only finitely many of the registers. Even though the memory capacity of a RAM is unlimited, any computation described by a RAM program will access only finitely much data, unless, of course, the computation described never terminates.
In case of a non terminating computation, the amount of data accessed at any given time is finite. Each instruction may have a label, where label names are chosen from the list: `N0,N1,....` Each instruction is of one of the following seven types:

1. `INC Ri`: Increments the contents of the register `Ri`.
2. `DEC Ri`: Decrements the contents of the register `Ri`.
3. `CLR Ri`: 'Clear's `Ri`, which now holds 0.
4. `Ri←Rj`: `Ri` 'becomes' or 'gets' `Rj`; replaces the contents of `Ri` with those of `Rj`, whose contents (the source) aren't changed.
5. `JMP Nix`: If `x` is `a`, then the next instruction to execute is the closest *preceding* instruction with label `Ni`. If `x` is `b` then the next instruction to execute is the closest *following* instruction with label `Ni`. The `a` stands for 'above' and the `b` for 'below.' *This unusual convention allows for the pasting together of programs without paying attention to instruction labels.* This instruction can be called an 'unconditional jump': the control will be transferred to the instruction labeled `Ni` above or below the current instruction.
6. `Rj JMP Nix`: Perform a `JMP` instruction as above if register `Rj` contains a 0. This instruction can be called a 'conditional jump.' If `Rj` does not contain 0, then the control simply 'falls through' to the next instruction in the program's text.
7. `CONTINUE` Do nothing.

Definition 1 (RAM Program)**.** A RAM program is a finite sequence of instructions such that each `JMP` instruction (conditional or otherwise) has a valid destination, e.g., the label referred to in the instruction exists, and the final statement is a `CONTINUE`.

Definition 2 (Halting)**.** A RAM program halts if and when it reaches the final `CONTINUE` statement.

Definition 3 (A RAM Computatble Function)**.** A RAM program $\mathcal{P}$ computes a partial function $\phi(x_1, x_2, \ldots, x_n)$ of n arguments, if and only if

1. $\mathcal{P}$ starts with $x_1, x_2, \ldots, x_n$ in registers $R_1, R_2, \ldots, R_n$ respectively,

2. All other registers used by $\mathcal{P}$ contain 0, and
3. $\mathcal{P}$ halts only if $\phi(x_1, x_2, \ldots, x_n)$ is defined and R_1 contains the value $\phi(x_1, x_2, \ldots, x_n)$.

A partial function is *RAM-computable* if some RAM program computes it.

1.2 Partial Recursive Functions

Base Functions

Not all of these need be 'functions', but are defined as such to keep the notation consistent.

$$\begin{aligned} &\text{(Zero)} && Z(x) = 0 \ \forall x \\ &\text{(Successor)} && S(x) = x + 1 \ \forall x \\ &\text{(Projection or Selection)} && U_j^n(x_1, x_2, \ldots, x_j, \ldots, x_n) = x_j \end{aligned} \tag{1.1}$$

Reflection 3 (No Identity Function!)**.** Interestingly, he doesn't define the identity function, $I(x) = x$, as a base function. However, $I = U_1^1$. His thrust may be on defining more generic (and hence more powerful) functions.

Definition 4 (Functions defined by 'primitive' recursion)**.** A function f of $n + 1$ arguments is defined by *primitive recursion* from 1) g, a function of n arguments, and 2) h, a function of $n + 2$ arguments, if and only if for each[1] $x_1, x_2, \ldots, x_n$:

$$\begin{aligned} f(x_1, x_2, \ldots, x_n, 0) &= g(x_1, x_2, \ldots, x_n) \\ f(x_1, x_2, \ldots, x_n, y + 1) &= h(x_1, x_2, \ldots, x_n, y, f(x_1, x_2, \ldots, x_n, y)) \end{aligned} \tag{1.2}$$

For the $n = 0$ case (i.e., functions applied to zero arguments[2]), we adopt the convention that a function of 0 arguments is a *constant.*

Box 2: What is Primitive Recursion?

*The recursion of the above definition is 'primitive' because the value $f(x_1, x_2, \ldots, x_n, y)$ can be determined from the value of $f(x_1, x_2, \ldots, x_n, z)$ for some $z < y$ ("looking back"). **Forms of recursion that are not primitive will be discussed extensively later in the book**[a].*

[a] "Non-primitive' recursive functions? I am curious!

— Carl Smith

Reflection 4 (General vs. Concrete)**.** This definition [1.2] appears too complicated at first. A typical definition of a recursive function usually refers to the hackneyed 'factorial' (factorial : $\mathbb{N} \to \mathbb{N}$) that is applied to a nonnegative integer:

$$\text{factorial}(x) = \begin{cases} 1, & \text{if } n = 0 \\ x \times \text{factorial}(x - 1), & \text{otherwise} \end{cases}$$

This is typical in formal sciences, especially math and computer science. We tend to *generalize* operations.

[1] I think he means for every sequence of n arguments, $x_1, x_2, \ldots, x_n$, not for each argument like x_1.
[2] Many programming languages call such functions `void`.

In time, we become better at finding a *level of generality* that balances flexibility with complexity. In this case, to define f in terms of itself, we also define g and h. To keep them generic, we consider functions with several arguments.

How does one *conceive* a definition such as [4]? I believe they start with some definition and refine it to a sufficient level of generality. Experience may be their guide to judge that level. With functions, their *arguments* and *return values* are all we have. Using that effectively may have also helped such a conception. $f : \mathbb{N}^{n+1} \to \mathbb{N}$ is defined by cases:

1. If the last argument is 0, f returns what $g : \mathbb{N}^n \to \mathbb{N}$, applied to the remaining (n) arguments, returns.
2. Otherwise, f returns what $h : \mathbb{N}^{n+2} \to \mathbb{N}$, applied to the remaining ones (n) and two carefully chosen arguments (that is, a total of $n + 2$ arguments), returns. In this case, the calculation of the last argument to h requires a recursive application of f.

Nonetheless, it feels mysterious at first.

Definition 5 (Functions defined by 'composition')**.** A function f of n arguments is defined by *composition* from 1) g, a function of m arguments, and 2) functions $h_1, h_2, \ldots, h_m$, each of n arguments, if and only if, for each $x_1, x_2, \ldots, x_n$:

$$f(x_1, x_2, \ldots, x_n) = g(h_1(x_1, x_2, \ldots, x_n), h_2(x_1, x_2, \ldots, x_n), \ldots, h_m(x_1, x_2, \ldots, x_n)) \tag{1.3}$$

The $m = 1$ case yields the traditional composition scheme, e.g., $f(x) = g(h(x))$.

Definition 6 (Class of primitive recursive functions)**.** The class of primitive recursive functions is the smallest class of functions containing all the base functions that is closed under the operations of primitive recursion and composition.

Examples

Reflection 5 (Creating functions from the 'base functions')**.** The book provides a definition of a function that adds its two arguments. Starting from the definition [4] and base functions [1.1], I will try to *derive* it here. Let's denote the function as A which returns an integer when applied to two integer arguments, x, y.

From equation [1.2], it follows that

$$\begin{aligned} A(x, 0) &= g(x) \\ A(x, y + 1) &= h(x, y, A(x, y)) \end{aligned} \tag{1.4}$$

Since our function A applies to two arguments, **our job is to find definitions of g (applies to one argument) and h (applies to three arguments) so that A returns the sum of its arguments**.

From the first of these equations, it is sufficiently clear that g is the identity function, $g = U_1^1$, which returns its argument when applied to any argument: $g(x) = x$.

If $y \neq 0$, applying A results in at least one more application of A. This process continues until y becomes zero, when applying A results in applying g, which returns the first argument (x). We say that, at that point, the (primitive) recursion 'bottoms out.' We can visualize applications of h thus:

$$
\begin{aligned}
&1) A(x,y) = h(x, y-1, A(x,y-1)) \\
&\qquad\qquad \vert \\
&\quad 2) h(x, y-2, A(x,y-2)) \\
&\qquad\qquad \vert \\
&\quad 3) \ h(x, y-3, A(x,y-3)) \\
&\qquad\qquad \vert \\
&\qquad\quad \cdots \\
&\qquad\qquad \vert \\
&\quad y) \ h(x, 0, A(x,0)) = h(x,0,x)
\end{aligned}
$$

Therefore, one application of A to two arguments, x, y, results in y applications of h; in each application of h, the last argument is decremented. What should h return in general then?

If h returned the successor of its last argument, every application of h simply counts up from x, y times! Therefore, h is just the successor of its last argument: $h(x,y,z) = S(z)$. In general, using our base functions:

$$
\begin{aligned}
g(x) &= x \\
h(x_1, x_2, x_3) &= S(x_3) \\
&A \text{ is now defined by primitive recursion:} \\
A(x_1, 0) &= g(x_1) \\
A(x_1, x_2 + 1) &= h(x_1, x_2, A(x_1, x_2))
\end{aligned}
\tag{1.5}
$$

Reflection 6 (Defining multiplication, M, by primitive recursion and composition)**.**

$$
\begin{aligned}
M(x_1, 0) &= g(x_1) \\
M(x_1, x_2 + 1) &= h(x_1, x_2, M(x_1, x_2))
\end{aligned}
\tag{1.6}
$$

Define g, h.

Since the product of anything and 0 is 0, g is the zero function, which we simply write as 0.

It is similar to addition, but I needed some experimentation. When the recursion bottoms out, M returns what g returns (i.e., 0). If h returns the sum of its first and last arguments, things work out nicely. Consider $M(2,3)$:

$$
\begin{aligned}
M(2,3) &= h(2,2,M(2,2)) \\
&= h(2,2,h(2,1,M(2,1))) \\
&= h(2,2,h(2,1,h(2,0,M(2,0)))) \\
&= h(2,2,h(2,1,h(2,0,0))) \\
&= h(2,2,h(2,1,2)) \\
&= h(2,2,4) \\
&= 6
\end{aligned}
\tag{1.7}
$$

Therefore,

$$
\begin{aligned}
g(x) &= 0 \\
h(x_1, x_2, x_3) &= A(x_1, x_3) \quad \text{See equation [1.5]} \\
&M \text{ is now defined by primitive recursion:} \\
M(x_1, 0) &= 0 \\
M(x_1, x_2 + 1) &= h(x_1, x_2, M(x_1, x_2))
\end{aligned}
\tag{1.8}
$$

Indeed, one can start mechanically[a] to define new functions from simple, yet general (and hence powerful) base functions (See equation [1.1]), primitive recursion (See definition [4]), and composition

(See definition [5])!

[a]Some creativity is also needed.

We can now define a very large class of functions. This class is large enough to include all the functions that we can actually compute today. In fact, all the functions we are likely to ever be able to compute are included in the class of primitive recursive functions (which is closed under primitive recursion and composition).

— Carl Smith

Reflection 7 (The predecessor function)**.** We already have a successor function, S (See equation [1.1]). Let's define a predecessor function, PR. The only peculiarity is that the predecessor of any number less than or equal to 0 is 0. Such subtraction where a negative number is *never* produced is called "proper subtraction".

We start with primitive recursion: PR is applied to one argument, therefore, $n = 0$, i.e., g is a 'constant' (applies to 0 arguments) and h applies to 2 arguments:

$$\begin{aligned} PR(0) &= 0 \\ PR(x+1) &= h(x, PR(x)) \end{aligned} \tag{1.9}$$

Our task is the same: Define h.

Applying P to a familiar number like 2, we get:

$$PR(2) = h(1, PR(1)) = h(1, h(0, PR(0))) = h(1, h(0, 0))$$

This should return 1. That means, h should just return the first argument. It just works!

$$PR(2) = h(1, PR(1)) = h(1, h(0, PR(0))) = h(1, h(0, 0)) = h(1, 0) = 1$$

Here is how we define the predecessor function by primitive recursion:

$$\begin{aligned} h(x_1, x_2) &= x_1 \\ PR(0) &= 0 \\ PR(x+1) &= h(x, PR(x)) \end{aligned} \tag{1.10}$$

Functions are great. Their syntax can result in complicated expressions, however. Therefore, the book adopts a more familiar or succinct notation for the reminder of our discussion:

- Since the projection function *selects and returns the j^{th} argument*, we simply make it return x_j, instead of $U_j^n(x_1, x_2, \ldots, x_n)$.
- $x + y$ is preferred to $A(x, y)$.
- $x \times y$ is preferred to $M(x, y)$.
- Proper subtraction (the one that never produces a negative number) is shown by the $\dot{-}$ symbol[3]: $x \dot{-} y$.

A few functions useful for testing conditions are defined next. However, the author does not provide English names for them[4]. I'll reuse his notation for these functions and provide the informal and formal definitions (using primitive recursion) for them.

[3]Macro '\dotdiv' from the `mathabx` package.

[4]I'll try to name them, hopefully succinctly.

The `logical true` function, sg

This function can be used to turn any function into a function with range $\{0, 1\}$. Informally:

$$sg(x) = \begin{cases} 0, & \text{if } x = 0 \\ 1, & \text{otherwise} \end{cases}$$

Formally:

$$\begin{aligned} h_2(x, y) &= S(Z(x)) \\ sg(0) &= 0 \\ sg(x+1) &= h_2(x+1, x+1) \end{aligned} \tag{1.11}$$

Reflection 8. Such formalism is perhaps necessary as a practice. For defining functions by primitive recursion, we define h in terms of *known functions* (which themselves may be defined by base functions, primitive recursion, and composition). We don't use 1 as it is not a previously defined function, but we could easily have done that: $f(x) = S(Z(x))$. In mathematics, we solve new problems by referring to already solved problems.

The `logical false` function, $\overline{sg}$

This function can also be used to turn any function into a function with range $\{0, 1\}$. Informally:

$$\overline{sg}(x) = \begin{cases} 1, & \text{if } x = 0 \\ 0, & \text{otherwise} \end{cases}$$

Formally:

$$\begin{aligned} h_2(x, y) &= Z(x) \\ \overline{sg}(0) &= S(Z(x)) \\ \overline{sg}(x+1) &= h_2(x+1, x+1) \end{aligned} \tag{1.12}$$

> *By interpreting* 0 *as 'false' and* 1 *as 'true' a* $0, 1$*-valued function represents a* predicate. *Logical predicates that can be represented by* $0, 1$*-valued primitive recursive functions are called* primitive recursive predicates.
>
> — Carl Smith

Reflection 9 (Other logical operations as primitive recursive functions)**.** I thought of defining other basic logical operations primitive-recursively. We can of course exploit logical formulas and tautologies.

1. `logical not`. We can either compose it from the `logical true` and `logical false` functions above (See equations [1.11], [1.12]), or we can define it from scratch by primitive recursion.

$$\begin{aligned} not(x) &= sg(\overline{sg}(x)) \\ &\text{Or,} \\ not(x) &= \overline{sg}(sg(x)) \end{aligned} \tag{1.13}$$

Or,

$$\begin{aligned} h_2(x_1, x_2) &= Z(x_1) \\ &\text{And,} \\ not(0) &= S(Z(x)) \\ not(x+1) &= h_2(x, not(x)) \end{aligned} \tag{1.14}$$

2. `logical (inclusive) or`. Some experimentation helps us arrive at:

$$\begin{aligned} g(x) &= sg(x) \\ h_3(x_1, x_2, x_3) &= S(Z(x_1)) \qquad \text{(Basically 1)} \\ &\text{And,} \\ or(x_1, 0) &= g(x_1) \\ or(x_1, x_2 + 1) &= h_3(x_1, x_2, or(x_1, x_2)) \end{aligned} \tag{1.15}$$

3. `logical and`.

$$\begin{aligned} g(x) &= Z(x) \\ h_3(x_1, x_2, x_3) &= sg(x_1) \\ &\text{And,} \\ and(x_1, 0) &= g(x_1) \\ and(x_1, x_2 + 1) &= h_3(x_1, x_2, and(x_1, x_2)) \end{aligned} \tag{1.16}$$

4. `logical xor`.

5. `logical (material) implication`.

The `proper subtraction` function, $\dot{-}$

We denote this[5] by $\mathsf{PSUB}(x_1, x_2)$, but adopt the more symbolic $x_1 \dot{-} x_2$, just like we adopt $x_1 + x_2$ instead of $A(x_1, x_2)$ for the `sum` function.

Informally,

$$x_1 \dot{-} x_2 = \begin{cases} x_1 - x_2, & \text{if } x_1 \geqslant x_2 \\ 0, & \text{otherwise} \end{cases}$$

Some experimentation shows that once the third argument of the h_3 function becomes zero (through recursion), it should stay 0. This suggests:

$$\begin{aligned} g(x) &= x \\ h_3(x_1, x_2, x_3) &= PR(x_3) \qquad \text{(See reflection[7])} \\ &\text{And,} \\ x_1 \dot{-} 0 &= g(x_1) \\ x_1 \dot{-} (x_2 + 1) &= h_3(x_1, x_2, (x_1 \dot{-} x_2)) \end{aligned} \tag{1.17}$$

The `difference` function, $|x_1 - x_2|$

We denote it by $\mathsf{DIFF}(x_1, x_2)$, but adopt the symbolic notation, $|x_1 - x_2|$.

Informally,

$$|x_1 - x_2| = \begin{cases} x_1 - x_2, & \text{if } x_1 \geqslant x_2 \\ x_2 - x_1, & \text{otherwise} \end{cases}$$

There are similarities between `DIFF` and `PSUB` (See [1.2]). This yields another informal definition of `DIFF`:

$$|x_1 - x_2| = (x_1 \dot{-} x_2) + (x_2 \dot{-} x_1)$$

Therefore, we can define `DIFF` by composition:

$$\mathsf{DIFF}(x_1, x_2) = \mathsf{A}(\mathsf{PSUB}(x_1, x_2), \mathsf{PSUB}(x_2, x_1))$$

How can we define it by primitive recursion (See definition [4])?

TBD

[5] Read 'x_1 *dot-minus* x_2'.

The `logical equality` function, ϵ

Informally,

$$\epsilon(x_1, x_2) = \begin{cases} 1, & \text{if } |x_1 - x_2| = 0 \\ 0, & \text{otherwise} \end{cases}$$

By composition, $\epsilon(x_1, x_2) = \overline{sg}(|x_1 - x_2|)$.

By primitive recursion (some experimentation helps; h_3 should return 1, when $x_1 - x_2 = 1$ and 0 otherwise),

$$\begin{aligned} g(x) &= not(x) \qquad \text{See equation [1.14]} \\ h_3(x_1, x_2, x_3) &= \overline{sg}(x_1 - x_2 - 1) \\ &\text{And,} \\ \epsilon(x_1, 0) &= g(x_1) \\ \epsilon(x_1, x_2 + 1) &= h_3(x_1, x_2, \epsilon(x_1, x_2)) \end{aligned} \tag{1.18}$$

The `logical inequality` function, $\overline{\epsilon}$

Informally, $\overline{\epsilon}(x_1, x_2) = not(\epsilon(x_1, x_2))$.

Exercise 1.1 (The 'rm' function is primitive recursive)**.** *Show that $rm(x, y) =$ 'The remainder left after dividing x by y' is primitive recursive.*

Solution. The division algorithm states that for any two integers (for now, we consider nonnegative integers), $x, y(y \neq 0)$, there exist two integers, q, r, such that $x = by + r \mid 0 \leqslant r < y$.

We need to define a function $rm(x, y)$ by primitive recursion. It's a partial function. It is not defined for $y = 0$.

$$\begin{aligned} rm(0, y) &= 0 \\ rm(x + 1, y) &= h(x, y, rm(x, y)) \end{aligned} \tag{1.19}$$

In this case, we don't need g, or, if we need it, then it is the zero function, Z (See equation [1.1]): rm returns 0 when x is 0. We adopt the simpler syntax.

Small examples help. When the first argument of rm is nonzero, rm will be called recursively, every time with predecessor of the first argument; recursion eventually bottoms out when it becomes zero:

$$rm(3, 2) = h(2, 2, rm(2, 2)) = h(2, 2, h(1, 2, rm(1, 2))) = h(2, 2, h(1, 2, h(0, 2, rm(0, 2)))) = h(2, 2, h(1, 2, h(0, 2, 0)))$$

It becomes clear that $h(x, y, z)$ returns what $rm(z + 1, y)$ returns.

This gives the recursive definition as:

$$\begin{aligned} rm(0, y) &= 0 \\ rm(x + 1, y) &= rm(rm(x, y) + 1, y) \end{aligned} \tag{1.20}$$

The textbook answer (which uses S for the successor function and M for the multiplication function) has a different answer. I will explore more:

$$\begin{aligned} rm(0, y) &= 0 \\ rm(x + 1, y) &= h(x, y, rm(x, y)) \\ &= M(S(rm(x, y)), \overline{\epsilon}(S(rm(x, y)), y)) \end{aligned} \tag{1.21}$$

Is my answer (equation [1.20]) wrong? I hope not, although it does not follow *the form* of the primitive recursive functions (See definition [4]), but neither does the textbook answer (See equation [1.21]). My definition does make rm primitive recursive because the function is defined in terms of itself, allowing recursion.

Exercise 1.2 (The 'integer division' is primitive recursive)**.** *Show that integer division is primitive recursive. Make and state a convention about division by 0.*

Solution.

Let's denote it by:/

Reflection 10. I took inspiration from JavaScript, which defines anything divided by 0 to be a special literal, NaN, which is not a real number.

$$\begin{aligned} g(x) &= \mathsf{NaN} \\ h_3(x_1, x_2, x_3) &= ?? \\ &\text{And,} \\ x_1 \,/\, 0 &= g(x_1) \\ x_1 \,/\, (x_2 + 1) &= h_3(x_1, x_2, x_1 \,/\, x_2) \end{aligned} \tag{1.22}$$

Reflection 11. This is tricky! I feel stuck. $h_3(x_1, x_2, x_3) = 0$ if $x_1 \leqslant x_2$, but otherwise I am not sure yet.

After a while, I reminded myself of the grade-school maxim, "Division is repeated subtraction." Then there was a ray of hope, although it renders a recursive definition for h_3. Is that okay?

$$h_3(x_1, x_2, x_3) = \begin{cases} 1 + h_3(x_1 - x_2, x_2, x_3), & \text{if } x_1 \geqslant x_2 \\ 0, & \text{otherwise} \end{cases}$$

That feels convoluted. The function we seek is defined by cases thus:

$$x_1 \,/\, x_2 = \begin{cases} \mathsf{NaN}, & \text{if } x_2 = 0, \\ 0, & \text{else if } x_1 < x_2, \\ 1 + (x_1 - x_2) \,/\, x_2 & \text{otherwise} \end{cases} \tag{1.23}$$

Reflection 12. According to the definition [4], equation [1.23] may not be primitive recursive, but according to the box [1.2] (and like the factorial function (See reflection [4])), it is.

The book does not provide a solution to this problem; I'll need to ask someone to check.

That raises another question: Are functions, like equation [1.23], that clearly set up recurrence, *not* defined by primitive recursion?

TBD

Exercise 1.3 ($\lfloor\sqrt{x}\rfloor$). *Show that $f(x) =$ 'The largest integer less than or equal to $\sqrt{x}$' is primitive recursive.*

Solution. Let me first write down the *skeleton*. We don't need g because $f(0)$ is just 0. We may also do $g(x) = Z(x) = 0$:

$$\begin{aligned} h_2(x_1, x_2) &= ?? \\ &\text{And,} \\ f(0) &= Z(x) \\ f(x_1 + 1) &= h_2(x_1, f(x_1)) \end{aligned} \tag{1.24}$$

We need to define h_2.

Reflection 13. Hmm. Tricky again! The floor function is introduced because $\sqrt{x}$ may not be integral or rational; flooring converts the argument into an integer. But the h_2 function seems elusive.

Reflection 14 (Patience is key, but ...). The definition of h_2 is still elusive...
I made a table of applications of h_2 to small arguments. That helps!

Table 1.1: Applications of f, h_2 to a Few Small Arguments. What is h_2?

f	h_2	h_2 **Simplified**	h_2 **Return**
$f(9)$	$h_2(\sqrt{8}, \lfloor\sqrt{8}\rfloor)$	$h_2(2.82, 2)$	**3**
$f(8)$	$h_2(\sqrt{7}, \lfloor\sqrt{7}\rfloor)$	$h_2(2.64, 2)$	2
$f(7)$	$h_2(\sqrt{6}, \lfloor\sqrt{6}\rfloor)$	$h_2(2.44, 2)$	2
$f(6)$	$h_2(\sqrt{5}, \lfloor\sqrt{5}\rfloor)$	$h_2(2.23, 2)$	2
$f(5)$	$h_2(\sqrt{4}, \lfloor\sqrt{4}\rfloor)$	$h_2(2, 2)$	2
$f(4)$	$h_2(\sqrt{3}, \lfloor\sqrt{3}\rfloor)$	$h_2(1.73, 1)$	**2**
$f(3)$	$h_2(\sqrt{2}, \lfloor\sqrt{2}\rfloor)$	$h_2(1.41, 1)$	1
$f(2)$	$h_2(\sqrt{1}, \lfloor\sqrt{1}\rfloor)$	$h_2(1, 1)$	1
$f(1)$	$h_2(\sqrt{0}, \lfloor\sqrt{0}\rfloor)$	$h_2(1, 0)$	**1**
$f(0)$	NA	NA	NA, f returns 0

Initially, I guessed it incorrectly. But then, something struck me. I observed that $h_2(x_1, x_2)$ is a step function returning x_2, stepping up to $x_2 + 1$ only when its first argument is the square root of a number one less than a perfect square (e.g., $\sqrt{8}, \sqrt{3}$), and its second argument is always an integer. Here is an attempt to define it:

$$h_2(x_1, x_2) = \begin{cases} x_2, & \text{if } (x_2+1)^2 - (x_1)^2 > 1 \\ x_2 + 1, & \text{otherwise} \end{cases} \tag{1.25}$$

This appears to work. And we know how to convert such functions into boolean functions (using $sg, \overline{sg}$).

We assume a function, sq, which squares its argument: $\text{sq}(x) = x \times x$. Since multiplication is primitive recursive, sq is so too.

$$\begin{gathered} h_2(x_1, x_2) = x_2 + \overline{sg}(\text{sq}(x_2) - \text{sq}(x_1) - 1) \\ \text{And,} \\ f(0) = Z(x) \\ f(x_1 + 1) = h_2(x_1, f(x_1)) \end{gathered} \tag{1.26}$$

Therefore, $f(x) =$ 'The largest integer less than or equal to $\sqrt{x}$' is primitive recursive.

Exercise 1.4. *Suppose that f is primitive recursive.*
Show that

$$\sum_{y \leqslant z} f(x_1, x_2, \ldots, y) = f(x_1, x_2, \ldots, 0) + \cdots + f(x_1, x_2, \ldots, z) \tag{1.27}$$

is primitive recursive.

Reflection 15 (Notational Fastidiousness ...). Equation [1.29] is fine, but I will employ a more standard notation:

$$\sum_{y=0}^{z} f(x_1, x_2, \ldots, y) = f(x_1, x_2, \ldots, 0) + \cdots + f(x_1, x_2, \ldots, z) \tag{1.28}$$

We are asked to prove that a function that returns a finite sum of applying a primitive recursive function to various arguments is also primitive recursive.
Informally, since $f, +$ are primitive recursive, by definition, $f_2(x_1) = f(x_1, 0) + f(x_1, 1)$ is primitive

recursive, but we have to prove it rigorously.

I need to write down the rigorous version of the observation above.

TBD

Exercise 1.5. *Suppose that f is primitive recursive.*
Show that

$$\Pi_{y=0}^{y\leqslant z} f(x_1, x_2, \ldots, y) = f(x_1, x_2, \ldots, 0) \times f(x_1, x_2, \ldots, 1) \times \cdots \times f(x_1, x_2, \ldots, z) \tag{1.29}$$

is primitive recursive.

Reflection 16. Informally, since $f, \times$ are primitive recursive, by definition, $f_2(x_1) = f(x_1, 0) \times f(x_1, 1)$ is primitive recursive, but we have to prove it rigorously.

TBD

Exercise 1.6. *Suppose that $g_1, g_2,$ and , g_3 are primitive recursive functions of n arguments, and that h_1 and h_2 are primitive recursive predicates of n arguments. Show that f, a function of n arguments defined as follows, is primitive recursive:*

$$f(x_1, x_2, \ldots, x_n) = \begin{cases} g_1(x_1, x_2, \ldots, x_n), & \text{if } h_1(x_1, x_2, \ldots, x_n), \\ g_2(x_1, x_2, \ldots, x_n), & \text{else if } h_2(x_1, x_2, \ldots, x_n) \text{ and not } h_1(x_1, x_2, \ldots, x_n), \\ g_3(x_1, x_2, \ldots, x_n), & \text{otherwise.} \end{cases} \tag{1.30}$$

Solution.

Proof. We prove this by exhaustion.

Since h_1, h_2 return either `true` or `false` on every input, by multiplication theorem, the number of choices of values of h_1, h_2 is 4. The following table emerges:

Table 1.2: f is Either g_1, g_2, or g_3

h_1	h_2	f
`true`	`true`	g_1
`true`	`false`	g_1
`false`	`true`	g_2
`false`	`false`	g_3

There is no input $(x_1, x_2, \ldots, x_n)$ such that when f is applied to it, the result is not one of g_1, g_2, or g_3. Thus, in any case, f returns what either of g_1, g_2, or g_3 returns. Since those functions are primitive recursive, in effect, f is also primitive recursive.

■

Exercise 1.7. *State and prove the n-ary version of the previous exercise (1.6).*
You start with n primitive recursive functions and $n - 1$ primitive recursive predicates. The resulting scheme is called 'course of values' recursion.

Solution. First, we state the problem: Suppose that $g_1, g_2, g_3, \ldots, g_n$ are n primitive recursive functions of m arguments, and that $h_1, h_2, \ldots, h_{n-1}$ are $n-1$ primitive recursive predicates of m arguments. Show that f, a function of m arguments defined as follows, is primitive recursive[6]:

$$f(x_1, x_2, \ldots, x_m) = \begin{cases} g_1(x_1, x_2, \ldots, x_m), & \text{if } h_1(x_1, x_2, \ldots, x_m), \\ g_2(x_1, x_2, \ldots, x_m), & \text{else if } h_2(x_1, x_2, \ldots, x_m) \text{ and not } h_1(x_1, x_2, \ldots, x_m), \\ g_3(x_1, x_2, \ldots, x_m), & \text{else if } h_3(x_1, x_2, \ldots, x_m) \text{ and not } h_2(x_1, x_2, \ldots, x_m) \\ & \text{and not } h_1(x_1, x_2, \ldots, x_m) \\ \vdots & \vdots \\ g_{n-1}(x_1, x_2, \ldots, x_m), & \text{else if } h_{n-1}(x_1, x_2, \ldots, x_m) \text{ and not } h_i(x_1, x_2, \ldots, x_m) \\ & \forall i, 1 \leqslant i \leqslant n-2, \\ g_n(x_1, x_2, \ldots, x_m), & \text{otherwise.} \end{cases} \tag{1.31}$$

Proof. We use a direct argument. To improve the readability of the notation, we will not specify the arguments of f, any of the g functions, and any of the h predicates. For example, we'll denote $f(x_1, x_2, \ldots, x_m)$ as $f()$.

There are n g functions (all primitive recursive) and $n-1$ h predicates (all primitive recursive).

We want to show that our scheme of applying the $n-1$ predicates *in order* divides *all* possible inputs $(x_1, x_2, \ldots, x_m)$ into n groups.

h_1 divides all inputs into two groups: all members of one group (say, `1T`) return `true`, and those of the other group (say, `1F`) return `false`. At this point, the number of groups is $2 = 1 + 1$, that is, one more than the number of predicates applied so far.

If we need to apply h_2 to some input (because applying h_1 to it returns `false`), the `1F` group is divided into two groups in a similar way: `2T` and `2F`. Again, at this point, the number of groups is $3 = 2 + 1$, that is, one more than the number of predicates applied so far. If we had no more predicates, the process stops. Applying f to any input now would result in applying g_1, g_2, or g_3 to it.

The 'process' ensures that having to apply a new predicate (which increments the number of predicates) also increments the number of groups for the 'universe of inputs'. $n-1$ predicates create n groups. Inductively, this always remains true. f must therefore be, in effect, one of $g_1, g_2, \ldots, g_n$.

■

Exercise 1.8. *Show that the primitive recursive predicates are closed under the operations of* `and`, `or`, `not`, *and* `implication`.

Reflection 17. The so-called 'closure property' is applied to certain sets. For example, we say that the set of natural numbers, $\mathbb{N}$, is closed under addition: $a + b$ returns a number that belongs to $\mathbb{N}$, the set of all primitive recursive functions is closed under the operation of forming functions by primitive recursion and composition.

What does he mean by 'set of primitive recursive functions is closed under `and`'? Consider two primitive recursive predicates, f_1, f_2. Then, (f_1 `and` f_2) returns either 0 or 1, not another predicate.

TBD

The book now discusses in more detail one of its core premises: datatype of variables over which functions are defined does not matter much. As it says in its Preface, it is only for convenience that programming languages provide various datatypes.

Two passing examples are given where we have functions with 1) variables that are strings of 0's and 1's, and 2) variables that are pairs of natural numbers. In each case, the corresponding definitions (See equation [1.1], definitions [4], [5]) will need to be stated.

6

Definition 7 (Functions on Strings of 0's and 1's)**.** We collect a few mini-definitions concerning strings under this umbrella definition. The *variable* is a string of 0's and 1's denoted by x.

An empty string ϵ is defined as a string of zero length. It serves the purpose for strings that 0 serves for natural numbers.

Juxtaposition denotes concatenation of strings.

The base function 'erase' denoted by $E(x)$ erases its argument: $E(x) = \epsilon$.

The base function 'concat 0' denoted by $S_0(x)$ appends a '0' to its argument: $S_0(x) = x0$.

The base function 'concat 1' denoted by $S_1(x)$ appends a '1' to its argument: $S_1(x) = x1$.

The projection function, as usual, selects the said argument: $U_j^n(x_1, x_2, \ldots, x_j, \ldots, x_n) = x_j$.

Functions of 01-strings are defined by composition in a manner similar to the functions of natural numbers defined by composition (See definition [5]).

A function f on n strings is defined by primitive recursion of three functions of strings, g of $n-1$ strings and h_0, h_1 of $n+1$ strings, as follows:

$$\begin{aligned} f(\epsilon, x_2, \ldots, x_n) &= g(x_1, x_2, \ldots, x_n) \\ f(y0, x_2, x_3, \ldots, x_n) &= h_0(y, f(y, x_2, x_3, \ldots, x_n), x_2, x_3, \ldots, x_n) \\ f(y1, x_2, x_3, \ldots, x_n) &= h_1(y, f(y, x_2, x_3, \ldots, x_n), x_2, x_3, \ldots, x_n) \end{aligned} \tag{1.32}$$

Reflection 18. It does not matter, but why change the order and number of arguments? Isn't the following definition that aligns better with the definition of primitive recursive functions of natural numbers (See definition [4]) equivalent (g takes n strings, f takes $n+1$, and h_0, h_1 take $n+2$)?

$$\begin{aligned} f(x_1, \ldots, x_n, \epsilon) &= g(x_1, x_2, \ldots, x_n) \\ f(x_1, x_2, \ldots, x_n, y0) &= h_0(x_1, x_2, \ldots, x_n, y, f(x_1, x_2, \ldots, x_n, y)) \\ f(x_1, x_2, \ldots, x_n, y1) &= h_1(x_1, x_2, \ldots, x_n, y, f(x_1, x_2, \ldots, x_n, y)) \end{aligned} \tag{1.33}$$

Exercise 1.9. *Show, using the full formalism, that the functions $dell$ and con below are primitive recursive:*

$$dell(x) = \begin{cases} \epsilon, & \text{if } x = \epsilon, \\ x \text{ with the last character (0 or 1) removed}, & \text{otherwise.} \end{cases} \tag{1.34}$$

$$con(x, y) = xy$$

Solution.

Proof. To prove that $dell$ is primitive recursive, we need to define the functions g, h_0, h_1, such that:

$$\begin{aligned} dell(\epsilon) &= g() \\ dell(y0) &= h_0(y, dell(y)) \\ dell(y1) &= h_1(y, dell(y)) \end{aligned} \tag{1.35}$$

From the first equation above, g is the empty string ϵ. Functions h_0, h_1 just choose their first argument, y:

$$\begin{aligned} g(x) &= \epsilon \\ h_0(x, y) &= x \\ h_1(x, y) &= x \end{aligned} \tag{1.36}$$

To prove that con is primitive recursive, we need to define the functions g, h_0, h_1, such that:

$$\begin{aligned} con(x, \epsilon) &= g(x) \\ con(x, y0) &= h_0(x, y, con(x, y)) \\ con(x, y1) &= h_1(x, y, con(x, y)) \end{aligned} \tag{1.37}$$

It follows from the above that $g(x)$ is just the identity function: $g(x) = x$.

Some experimentation shows that h_0, h_1 return the 0- and 1-successor of the third argument respectively:

$$\begin{aligned} g(x) &= x \\ h_0(x, y, z) &= S_0(z) \\ h_1(x, y, z) &= S_1(z) \end{aligned} \tag{1.38}$$

■

Exercise 1.10. *Define the primitive recursive functions over variables that are pairs of natural numbers.*

Solution.

Reflection 19. We must define a pair of natural numbers denoted as (x, y). Is (x, y) different from (y, x)? Our choice dictates the operations we'll designate as 'base functions.' It's a pair of natural numbers, so they retain some properties of natural numbers.

After some consideration, we adopt an *ordered* pair. Therefore, we denote the ordered pair $((\mathbb{N}, \mathbb{N}))$ of natural numbers as (x, y). Functions on pairs may return pairs (pairs can be said to be 'closed' under those functions), natural numbers, or something else.

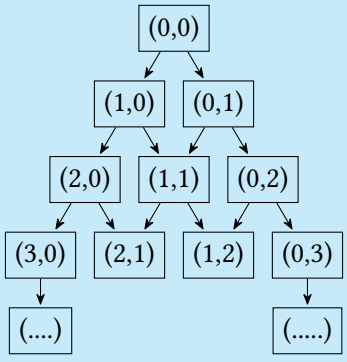
It's perhaps straightforward to designate a zero, two successors, and two projections of a pair:

$$\begin{aligned} Z(x, y) &= (0, 0) \\ S_1(x, y) &= (x + 1, y) \\ S_2(x, y) &= (x, y + 1) \\ U_1(x, y) &= x \\ U_2(x, y) &= y \end{aligned} \tag{1.39}$$

Composition can be similar: A function f is defined by composition of g, h as: $f(x, y) = g(h(x, y))$, where f, g, h accept and return a pair.

Reflection 20. I struggled with clarity here. Then, to some disgrace, I played chess. Not everything, except precious time of course, was lost. Fortunately, I recovered in time from the apparent loss of focus. Then, I made a deal with my daughter: I won't fall into the 'unplanned chess abyss' for more than 4 hours in the next 5 weeks. Let's see if I follow through.

The struggle is with whether a pair of natural numbers makes sense *only* as a pair. Is a pair of natural numbers a partition of a natural numbers? I favor not. A pair is just an ordered 2-tuple. This vision leads to a graph of pairs like the sequence of natural numbers. We use the two successor functions for each node:



The domain of all the 'pair functions' is this infinite graph (which is also the set $\{(a, b) \mid a \in \mathbb{N}, b \in \mathbb{N}\}$) of pairs, just like how the domain of all the 'natural number functions' is $\mathbb{N}$.

Let us first consider the case where we have four functions: g taking one pair, f two, and h_1, h_2 three. We define f primitive-recursively:

$$\begin{aligned} f((x_0, y_0), (0, 0)) &= g(x_0, y_0) \\ f((x_0, y_0), (x + 1, y)) &= h_1((x_0, y_0), (x, y), f((x_0, y_0), (x, y))) \\ f((x_0, y_0), (x, y + 1)) &= h_2((x_0, y_0), (x, y), f((x_0, y_0), (x, y))) \end{aligned} \tag{1.40}$$

Exercise 1.11. *Prove that a primitive recursive function is defined on* ***all its arguments****.*

Hint: Use a structural induction over the number of applications of primitive recursion and composition used to define a function.

Solution.

Reflection 21. Initially, I was confused about the question. Reading further, however, made me realize that the question is about whether a primitive recursive function is defined for every member of its domain. Such relations are also called *total*. A *total* satisfies the '$\geqslant 1$ arrows out' property, a function satisfies the '$\leqslant 1$ arrows out' property. Thus, for a relation to be a 'total function,' it must satisfy the '$= 1$ arrows out' property. In other words, our relation (or function) must be *defined* for every member of its domain.

Needn't (from the definition [4]) g, h be total for f to be a total? I guess we assume that.

Setting up the recurrence shouldn't be very hard; I'll revisit this one.

TBD

Reflection 22. The next discussion in the book is somewhat confusing. He says several things that don't readily align with each other. He uses the above exercise to state that primitive recursive functions are total (they are defined for all their inputs) and shows a simple RAM program that does not halt:

```
N1 INC R1
   JMP N1a
   CONTINUE
```

This helps him conclude that there are RAM programs that compute functions that are not primitive recursive (I am uncomfortable with this).

But I understand that he's trying to show that primitive recursive functions and RAM programs are equivalent models of computation.

This discussion apparently 'motivates' him to define yet another operator to define functions from functions. This operator is similar to '`while` loops' in programming languages. I need to analyze this closely.

Definition 8 (Functions defined by 'Minimalization')**.** A function f of n arguments is defined by minimalization from a function h of $n + 1$ arguments if $f(x_1, x_2, \ldots, x_n)$ is the least y such that

1) $h(x_1, x_2, \ldots, x_n, y) = 0$, and

2) $h(x_1, x_2, \ldots, x_n, z)$ is defined for every $z < y$.

Such a y may not exist, in which case, $f(x_1, x_2, \ldots, x_n)$ is undefined.

Reflection 23. This definition requires that we evaluate

$h(x_1, x_2, \ldots, x_n, 0)$,

$h(x_1, x_2, \ldots, x_n, 1)$,

$h(x_1, x_2, \ldots, x_n, 2)$,

....

(in other words, we 'loop over' the last argument to h and find the y (if it exists) that satisfies both conditions).

How does one come up with such a 'minimalization' definition (to rectify the above limitation of primitive recursion) in the first place?

I want to clarify this a bit more. How can I define a single variable function $f(x) \mid x \in \mathbb{N}$ from minimalization of another function $h(x, y) \mid x, y \in \mathbb{N}$? What does it all mean?

Before that: 'Partial' recursive functions consist of base functions (Z, S, U) combined using the rules of defining new functions by primitive recursion, composition, and minimalization.

Definition 9 (Class of Partial Recursive Functions)**.** The class of partial recursive functions is the smallest class of functions containing the base functions (equation [1.1]) that is closed under the operations of primitive recursion (definition [4]), composition (definition [5]) and minimalization (definition [8]).

I skipped a few definitions and exercises!

TBD

> *The partial recursive functions appear to be quite different from the RAM computable functions. The RAM programs suggest an imperative implementation while a functional programming approach is more suitable for the partial recursive functions. We will prove that the RAM computable functions are precisely the partial recursive functions. Toward that goal, we present our first theorem.*
>
> — Carl Smith

Theorem 1. *Every partial recursive function is RAM computable.*

Proof. We'll skip the detailed proof, but outline the sketch here.

This proof is a structural induction argument. For the base case, we show that the RAM computable functions contain the base functions. The inductive step(s) start with some partial recursive functions that are assumed to be RAM computable and show that if they are combined via the operations of primitive recursion, composition and/ or minimalization, then the resultant function is RAM computable.

The induction is over the implicit structure of the partial recursive functions. Each partial recursive function, ψ, is defined in terms of other partial recursive functions. Then ψ will follow the functions used to define it in the ordering of the partial recursive functions. Hence, an ordering of the partial recursive functions is induced by their definitions.

The base functions (Z, S, U) form the beginning of the ordering. The implicit induction hypothesis assumes that all functions prior to ψ in this unspecified ordering are RAM computable. The induction step then uses the definition of ψ to show that ψ is RAM computable. There will be a case for each possible way that ψ could have been defined from partial recursive functions that appear before ψ in the ordering.

A RAM program for Z (the Zero function):

```
N0 CLR R1
  CONTINUE
```

A RAM program for S (the Successor function):

```
N0 INC R1
  CONTINUE
```

A RAM program for U_j^n (the Projection function):

```
N0 R1←Rj
  CONTINUE
```

Thus, all the primitive recursive base functions are RAM computable.

Suppose that f, a function of $n + 1$ arguments, is defined by primitive recursion from g and h. Suppose $\mathcal{P}_g$ is a RAM program computing g and $\mathcal{P}_h$ is a RAM program computing h. Suppose that neither $\mathcal{P}_g$ nor $\mathcal{P}_h$ references any register R_j for any $j \geqslant k$. Let $N1$ be a label that is not used in $\mathcal{P}_h$. Below is a program computing f.

Reflection 24. Smith tries to teach us how to stitch existing RAM programs together to write other RAM programs. We have all used the essential facility of 'defining' and 'calling' functions or procedures in higher-level programming languages. Smith's idea sounds similar, however, we must do it with the 7 instructions (see box [1.1]).

Our job is to write a RAM program using these instructions to calculate f, which is defined (definition [4]) primitive-recursively from g, h, given their RAM programs.

I am used to writing RAM programs in higher-level languages, where 'functions' are taken for granted. Here, Smith wants us to not assume that facility. Am I excited to write this? As an assignment, yes, but as a matter of practicality, not so much. This is even more so when you feel that your progress is slow.

I read the program. It is a well-written program; an excellent example of how much can be done using simple constructs! I changed my mind. I decided to analyze it properly.

It is a faithful reproduction of the definition [4]. It clearly demonstrates that recursion and iteration (realized by the so-called 'loops') are equivalent. Smith sets up the initial conditions, invokes $\mathcal{P}_g$, and then 'loops over' the last argument $(y+1)$ of f (which is also the penultimate argument of $\mathcal{P}_h$), while preserving the result of each invocation. One must be precise, but I understand the program logic.

Table 1.3: Primitive Recursion is RAM-computable

	$R_{k+1} \leftarrow R_1$	Save the n arguments to g, f, h
	$\vdots$	These n arguments are initially
	$R_{k+n} \leftarrow R_n$	placed in as many registers R_1 to R_n
	CLR R_{k+n+1}	Initialize the iteration number
	$R_k \leftarrow R_{n+1}$	The $n+1^{th}$ arg to f is $y+1$
		Treat it as the number of iterations
	P_g	Compute g; its arguments are in place
	$R_{k+n+2} \leftarrow R_1$	$\mathcal{P}_g$ places g's return value in R_1; save it
N1	R_k JMP N2b	Test for termination; R_k had preserved num iterations
	$R_1 \leftarrow R_{k+1}$	Restore the n arguments to g, f, h
	$\vdots$	
	$R_n \leftarrow R_{k+n}$	
	$R_{n+1} \leftarrow R_{k+n+1}$	$(n+1)^{th}$ argument of f
	$R_{n+2} \leftarrow R_{k+n+2}$	$(n+2)^{th}$ argument of h
	CLR R_{n+3}	Clear scratch registers
	$\vdots$	
	CLR R_{k-1}	
	P_h	Compute h; its arguments are in place
	$R_{k+n+2} \leftarrow R_1$	$\mathcal{P}_h$ places h's return value in R_1; save it
	INC R_{k+n+1}	Prepare for the next iteration
	DEC R_k	R_k had $y+1$ to begin with
	JMP N1a	
N2	CONTINUE	

Hence, the result of combining RAM computable functions by primitive recursion will always result in a RAM computable function. Now, continuing with the inductive step, we proceed to consider combining RAM programs by composition (See definition [5]).

Reflection 25. This one feels much easier: Arrange for a loop to 'load up' each of the m $\mathcal{P}_h$ programs that receive the n arguments in place. The results are collected in m registers. Then $\mathcal{P}_g$ is loaded and run, effectively calculating f.

■

Bibliography

[1] Paul R. Halmos. *I Want to be a Mathematician.* MAA Press, 2005.

[2] Carl H. Smith. *Recursive Introduction to the Theory of Computation.* Springer-Verlag, 1994.